nodes, we need a new generator, `d3.svg.diagonal`, which draws a curved line from one point to another.

**Listing 5.4 Callback function to draw a dendrogram**

```
var treeChart = d3.layout.tree();
treeChart.size([500,500])
     .children(function(d) {return d.values});

var linkGenerator = d3.svg.diagonal();

d3.select("svg")
   .append("g")
   .attr("id", "treeG")
   .selectAll("g")
   .data(treeChart(packableTweets))
   .enter()
   .append("g")
   .attr("class", "node")
   .attr("transform", function(d) {
          return "translate(" +d.x+","+d.y+")"
   });

d3.selectAll("g.node")
   .append("circle")
   .attr("r", 10)
   .style("fill", function(d) {return depthScale(d.depth)})
   .style("stroke", "white")
   .style("stroke-width", "2px");

d3.selectAll("g.node")
   .append("text")
   .text(function(d) {return d.id || d.key || d.content})

d3.select("#treeG").selectAll("path")
  .data(treeChart.links(treeChart(packableTweets)))
   .enter().insert("path","g")
   .attr("d", linkGenerator)
   .style("fill", "none")
   .style("stroke", "black")
   .style("stroke-width", "2px");
```

**Creates a diagonal generator with the default settings**

**Creates a parent <g#treeG> to put all these elements in**

**This time we'll create <g> elements so we can label them.**

**Like the pack layout, the tree layout computes the XY coordinates of each node.**

**Uses packableTweets and depthScale from the previous example**

**A little circle representing each node that we color with the same scale we used for the circle pack**

**A text label for each node, with the text being either the id, key, or content attribute, whichever the node has**

**The .links function of the layout creates an array of links between each node that we can use to draw these links.**

**Just like all the other generators**

Our dendrogram in figure 5.14 is a bit hard to read. To turn it on its side, we need to adjust the positioning of the `<g>` elements by flipping the x and y coordinates, which orients the nodes horizontally. We also need to adjust the `.projection()` of the diagonal generator, which orients the lines horizontally:

```
     linkGenerator.projection(function (d) {return [d.y, d.x]})
...

     .append("g")
...
     .attr("transform", function(d) {return "translate(" +d.y+","+d.x+")"});
```